

MANGHWA_VERSE

Documentation Technique Complète

HTML · CSS · JavaScript

Projet pédagogique — Esdras Joseph NGUIE GUEKO · 2025

Table des matières

PARTIE 1 — HTML — Structure des pages

- 1.1 Vue d'ensemble du projet
- 1.2 La barre de navigation (navbar)
- 1.3 La section héro (index.html)
- 1.4 Les cartes d'œuvres (cards)
- 1.5 Les pages mangas.html & manhwas.html
- 1.6 Le pied de page (footer)
- 1.7 Les attributs data-page et for (labels/modals CSS)

PARTIE 2 — CSS — Mise en forme et design

- 2.1 Architecture du fichier style.css
- 2.2 Navbar & hero
- 2.3 Grille de cartes
- 2.4 Responsive design

PARTIE 3 — JavaScript — Logique et interactivité

- 3.1 Pourquoi JavaScript côté client ?
- 3.2 La structure de données dataManghwa
- 3.3 Injection dynamique du HTML et des styles (UIHTML / UIStyles)
- 3.4 Pourquoi des modals ? Choix de conception
- 3.5 Ouverture et fermeture du modal (openModal)
- 3.6 Le carrousel de personnages
- 3.7 La table de correspondance titleToKey
- 3.8 Interception des clics (event delegation)
- 3.9 La barre de recherche
- 3.10 Gestion des erreurs d'image (onerror)

PARTIE 4 — Bonnes pratiques & pistes d'amélioration

- 4.1 Ce qui fonctionne bien
- 4.2 Points d'amélioration possibles

PARTIE 1 — HTML : Structure des pages

Le projet comporte 4 pages HTML : **index.html** (accueil), **mangas.html**, **manhwas.html** et **propos.html**. Chaque page partage la même feuille de style **style.css** et le même fichier **script.js**. Ce principe d'un script unique évite la duplication de code et garantit un comportement identique sur toutes les pages.

1.1 — Vue d'ensemble du projet

L'architecture du projet est intentionnellement simple (pas de framework, pas de build tool). Tout repose sur trois fichiers fondamentaux :

index.html	Page d'accueil avec hero et deux sections Old_Gen / New_Gen
mangas.html	Liste des mangas japonais
manhwas.html	Liste des manhwas coréens
propos.html	Page À propos du projet
style.css	Feuille de style globale partagée par toutes les pages
script.js	Logique JavaScript unique (modal, recherche, données)
image/	Dossier contenant toutes les couvertures et images de personnages

1.2 — La barre de navigation (navbar)

La navbar est copiée à l'identique dans chaque fichier HTML. C'est un choix pédagogique simple ; dans un projet de production on utiliserait un composant partagé (PHP include, framework JS, etc.).

```
<nav class="navbar">
<div class="logo">Manghwa_Verse</div>
<ul class="nav-links">
<a href="index.html" data-page="index.html">Accueil</a>
<a href="mangas.html" data-page="mangas.html">Mangas</a>
....
</ul>
</nav>
```

L'attribut **data-page** sur chaque lien est un *data attribute* HTML5 : il permet au JavaScript (ou CSS) d'identifier la page active et de surligner le lien correspondant. Il ne sert pas encore dans ce projet mais constitue une bonne pratique pour de futures évolutions.

Note : Note : l'élément est utilisé à tort comme conteneur des liens — les liens sont des <a> directs, pas des . C'est une erreur sémantique mineure sans impact visuel mais à corriger pour l'accessibilité.

1.3 — La section héro (index.html uniquement)

Le héro est la grande bannière d'accueil. Il contient le titre du site, un sous-titre descriptif et deux boutons d'appel à l'action.

```
<header class="hero" id="accueil">
<h1>Bienvenue sur Manghwa_Verse</h1>
<h2>Découvrez tous vos Mangas...</h2>
<div class="hero-buttons">
<a href="mangas.html" class="btn">Voir les Mangas</a>
<a href="manhwas.html" class="btn">Voir les Manhwas</a>
</div>
</header>
```

L'attribut `id="accueil"` permet un ancrage direct via l'URL (`#accueil`). Les balises sémantiques `h1/h2/h3` structurent la hiérarchie du contenu pour les moteurs de recherche et les lecteurs d'écran.

1.4 — Les cartes d'œuvres (.card)

Chaque œuvre est représentée par une carte cliquable. Le choix de la balise `<label>` (plutôt que `<div>` ou `<a>`) vient d'une ancienne approche CSS-only où le label déclenchait un `input:checkbox` caché pour ouvrir un modal sans JavaScript. Cette technique a depuis été remplacée par la gestion JS, mais les labels ont été conservés pour ne pas casser le markup existant.

```
<label for="modal-naruto" class="card">

<h3>Naruto</h3>
</label>
```

<code>for="modal-naruto"</code>	Attribut hérité de l'ancienne technique label/checkbox. Ignoré par le JS actuel.
<code>class="card"</code>	Classe CSS utilisée pour le style ET pour la détection des clics JS.
<code>img alt</code>	Texte alternatif essentiel pour l'accessibilité et le SEO.
<code>h3</code>	Titre de la carte, utilisé par le JS pour identifier l'œuvre via son texte.

1.5 — mangas.html & manhwas.html

Ces deux pages ont une structure identique : navbar + section avec un titre `h2` et un conteneur `.cards` rempli de cartes. La seule différence est le contenu des cartes et le titre de la section (`.top-mangas` vs `.top-manhwas`). Comme le `script.js` est le même, toutes les cartes de ces pages bénéficient automatiquement du modal et de la recherche.

1.6 — Le pied de page (footer)

Présent uniquement sur index.html. Il contient la mention de copyright et un avertissement légal précisant que le site est un projet pédagogique non lucratif et que les œuvres citées appartiennent à leurs auteurs.

1.7 — Attributs data-page et for

Les **data attributes** (data-*) sont des attributs HTML5 personnalisés. Ils permettent de stocker des informations directement dans le markup sans utiliser des classes ou ids détournés de leur usage. Ici data-page="index.html" pourrait être lu par JS pour ajouter une classe 'active' au lien de la page courante :

```
// Exemple d'usage possible
const current = window.location.pathname.split('/').pop();
document.querySelectorAll('[data-page]').forEach(link => {
  if (link.dataset.page === current) link.classList.add('active');
});
```

PARTIE 2 — CSS : Mise en forme et design

2.1 — Architecture du fichier style.css

Un seul fichier style.css est partagé par toutes les pages. Cette approche centralise tous les styles et évite les incohérences visuelles. Les styles injectés par script.js (UIStyles) viennent en complément pour les éléments créés dynamiquement (modal, recherche, carrousel).

2.2 — Navbar & hero

La navbar utilise **position: fixed** pour rester visible lors du scroll. Le hero utilise généralement un background-image avec un overlay sombre pour créer une ambiance immersive. Les boutons .btn ont un style cohérent avec le thème sombre du site.

2.3 — Grille de cartes

Le conteneur .cards utilise **display: flex** avec **flex-wrap: wrap** pour disposer les cartes en grille responsive. Dans script.js, les cartes reçoivent une largeur fixe forcée :

```
.card, .manga-item, .box {  
  max-width: 220px !important;  
  width: 220px !important;  
  flex: 0 0 220px !important; /* ne grandit/rétrécit pas */  
}
```

La valeur **!important** est nécessaire ici car ces styles sont injectés via JavaScript après le chargement du CSS, et ils doivent prendre le dessus sur les règles existantes. C'est un usage justifié de !important (surcharge dynamique ciblée).

2.4 — Responsive design

Une media query dans UIStyles adapte le modal aux petits écrans :

```
@media (max-width: 850px) {  
  .modal-layout { flex-direction: column; align-items: center; }  
  .modal-left { width: 240px; }  
  .modal-right h1 { font-size: 2rem; }  
}
```

En dessous de 850px, le layout passe de côte-à-côte (image + texte) à empilé verticalement, rendant le modal utilisable sur mobile.

PARTIE 3 — JavaScript : Logique et interactivité

3.1 — Pourquoi JavaScript côté client ?

Le projet n'a pas de serveur ni de base de données. Tout le contenu (titres, synopsis, personnages) est stocké directement dans `script.js`. C'est une approche valide pour un projet pédagogique statique : aucun hébergement spécial n'est nécessaire, les fichiers s'ouvrent directement dans un navigateur. JavaScript est utilisé pour :

- Afficher les informations détaillées d'une œuvre dans un modal (sans rechargement de page)
- Naviguer dans les personnages via un carrousel
- Filtrer les cartes en temps réel avec la barre de recherche
- Injecter dynamiquement le HTML et le CSS du modal (une seule fois, pour toutes les pages)

3.2 — La structure de données `dataManghwa`

C'est le cœur du projet : un objet JavaScript qui joue le rôle de **base de données côté client**. Chaque clé est le titre de l'œuvre en minuscules, et la valeur est un objet contenant toutes ses informations.

```
const dataManghwa = {
  "naruto": {
    title: "Naruto",
    image: "image/naruto.webp",
    synopsis: "Dans le village de Konoha...",
    episodes: "720",
    tomes: "72",
    saisons: "26",
    powerSystem: "Chakra",
    characters: [
      { name: "Naruto Uzumaki", img: "image/naruto_uzumaki_perso.jpg" },
      { name: "Sasuke Uchiha", img: "image/sasuke_perso.jpg" },
      // ...
    ]
  },
  // toutes les autres oeuvres...
};
```

Pourquoi un objet et non un tableau ? Les objets permettent un accès direct par clé en $O(1)$: `dataManghwa["naruto"]` est instantané, alors qu'un tableau nécessiterait une boucle de recherche. Les clés sont en minuscules pour simplifier la comparaison avec le texte des cartes (transformé en minuscules via `.toLowerCase()`).

3.3 — Injection dynamique du HTML et des styles

Plutôt que de copier le code du modal dans chaque fichier HTML, script.js l'injecte une seule fois à la fin du :

```
// Injection du HTML du modal et de la barre de recherche
document.body.insertAdjacentHTML('beforeend', UIHTML);

// Injection des styles CSS associés
const styleSheet = document.createElement('style');
styleSheet.innerText = UIStyles;
document.head.appendChild(styleSheet);
```

`insertAdjacentHTML('beforeend', ...)` est plus performant que `innerHTML+=` car il ne reparse pas tout le DOM existant. `createElement('style') + appendChild` crée une balise `<style>` dynamique dans le `<head>`, ce qui est la méthode standard pour injecter du CSS via JavaScript.

Note : Avantage : si on ajoute une 5e page HTML demain, il suffit d'y inclure script.js et elle aura automatiquement le modal et la recherche, sans toucher au HTML.

3.4 — Pourquoi des modals ? Choix de conception

Un **modal** (aussi appelé fenêtre de dialogue ou overlay) est une couche qui s'affiche par-dessus le contenu principal sans quitter la page. Plusieurs alternatives ont été envisagées :

Approche	Avantages	Inconvénients
Page dédiée (ex: naruto.html)	URL propre, SEO favorable	50+ fichiers HTML à créer et maintenir
Accordion/Toggle CSS	Sans JavaScript	Peu ergonomique, mise en page compliquée
Modal JS (choix retenu)	Un seul fichier HTML, expérience fluide, contenu riche	Nécessite JS, pas de lien direct vers une œuvre

Le modal a été choisi car il offre le meilleur équilibre entre richesse d'information (image grande, synopsis, stats, carrousel de personnages) et simplicité de maintenance (toutes les données dans un seul objet JS, un seul template HTML).

3.5 — Ouverture et fermeture du modal (openModal)

La fonction `openModal()` reçoit une clé (ex: "naruto"), récupère l'objet correspondant dans `dataManghwa`, remplit les éléments du DOM et rend le modal visible.

```
function openModal(workKey) {
const item = dataManghwa[workKey.toLowerCase()];
if (!item) return; // Sécurité : oeuvre inconnue → on ne fait rien

// Remplissage des champs
document.getElementById('modal-title').innerText = item.title;
document.getElementById('modal-img').src = item.image;
```



```

document.getElementById('modal-synopsis').innerText = item.synopsis;
// ... (épisodes, tomes, saisons, powerSystem)
// Génération du carrousel de personnages
track.innerHTML = '';
item.characters.forEach(char => {
const card = document.createElement('div');
card.className = 'character-card';
card.innerHTML = `<img src='${char.img}' ...><p>${char.name}</p>`;
track.appendChild(card);
});

modal.style.display = 'flex';
document.body.style.overflow = 'hidden'; // Bloque le scroll de la page
}

```

Trois façons de fermer le modal sont implémentées pour une UX optimale :

- Clic sur le bouton × (closeBtn.onclick)
- Clic sur le fond semi-transparent (modal.addEventListener click sur la cible directe)
- Touche Echap au clavier (document keydown, vérifie e.key === 'Escape')

3.6 — Le carrousel de personnages

Le carrousel affiche les personnages principaux sous forme de cartes défilantes. Il est implémenté en CSS (transform: translateX) contrôlé par JavaScript, sans librairie externe.

```

// Structure HTML du carrousel (injectée dans UIHTML)
<div class="carousel-container">
<button class="carousel-btn prev-btn">&#10094;</button>
<div class="carousel-track-container"> <!-- fenêtre visible -->
<div id="carousel-track" class="carousel-track"> <!-- piste qui se déplace -->
<!-- .character-card injectées dynamiquement -->
</div>
</div>
<button class="carousel-btn next-btn">&#10095;</button>
</div>

// Logique de défilement
let carouselPosition = 0; // Variable d'état globale

document.querySelector('.next-btn').onclick = () => {
const maxScroll = track.scrollWidth - track.parentElement.clientWidth;
if (carouselPosition > -maxScroll) {
carouselPosition -= 145; // 130px carte + 15px gap
track.style.transform = `translateX(${Math.max(carouselPosition, -maxScroll)}px)`;

```

```
}  
};
```

scrollWidth est la largeur totale de la piste (toutes cartes incluses). **clientWidth** est la largeur visible de la fenêtre. La différence donne le scroll maximum possible. **Math.max(..., -maxScroll)** empêche de dépasser la fin. La transition CSS (transition: transform 0.3s cubic-bezier...) rend le défilement fluide.

3.7 — La table de correspondance titleToKey

La table titleToKey fait le lien entre le texte visible sur les cartes HTML et les clés de l'objet dataManghwa. Elle est nécessaire pour deux raisons :

- Les titres dans le HTML peuvent contenir des fautes de frappe (ex: 'Drangon Ball', 'Swwordmaster')
- Un même titre peut avoir plusieurs formes (ex: 'City Hunter' et 'Nicky Larson' pointent vers la même clé)

```
const titleToKey = {  
  "naruto": "naruto",  
  "drangon ball": "dragon ball", // Faute de frappe dans le HTML  
  "nicky larsen": "city hunter", // Alias  
  "jojo's bizzarre adventure": "jojo's bizarre adventure", // Faute  
  // ...  
};
```

3.8 — Interception des clics (Event Delegation)

Plutôt que d'attacher un écouteur (addEventListener) sur chacune des 80+ cartes, on utilise la technique de **délégation d'événements** : un seul écouteur sur le document entier capte tous les clics et vérifie si la cible est une carte.

```
document.addEventListener('click', (e) => {  
  // .closest() remonte l'arbre DOM pour trouver le parent .card  
  const card = e.target.closest('.card, .manga-item, .box');  
  if (!card) return; // Clic en dehors d'une carte → on ignore  
  
  const rawText = card.innerText.toLowerCase().trim();  
  
  // Cherche la correspondance la plus longue dans titleToKey  
  let foundKey = null;  
  let bestMatchLength = 0;  
  
  for (const [title, key] of Object.entries(titleToKey)) {  
    if (rawText.includes(title) && title.length > bestMatchLength) {  
      foundKey = key;  
      bestMatchLength = title.length;  
    }  
  }  
  
  if (foundKey) openModal(foundKey);  
});
```

```
});
```

La recherche de la **correspondance la plus longue** est cruciale : sans elle, 'the breaker' serait matché avant 'the breaker new waves' et on obtiendrait un mauvais résultat. En gardant la clé dont le titre est le plus long parmi ceux qui correspondent, on garantit la précision maximale.

Note : Avantage de la délégation : les cartes ajoutées dynamiquement après le chargement de la page seraient automatiquement gérées, sans re-attacher d'écouteurs.

3.9 — La barre de recherche

La recherche filtre les cartes en temps réel à chaque frappe (événement 'input'). Elle compare le texte de chaque carte avec la saisie de l'utilisateur.

```
document.getElementById('manga-search').addEventListener('input', (e) => {
  const query = e.target.value.toLowerCase().trim();
  const cards = document.querySelectorAll('.card, .manga-item, .box');

  cards.forEach(card => {
    const title = card.innerText.toLowerCase();
    if (title.includes(query)) {
      card.style.setProperty('display', '', 'important'); // Affiche
    } else {
      card.style.setProperty('display', 'none', 'important'); // Masque
    }
  });
});
```

L'utilisation de `setProperty('display', '', 'important')` plutôt que `card.style.display = ''` est nécessaire car les styles !important du CSS ne peuvent être écrasés que par d'autres !important. Passer une chaîne vide "" réinitialise la propriété à sa valeur CSS originale.

La barre de recherche est positionnée en **position: fixed** en haut à droite et utilise une animation d'expansion au survol : elle part d'une icône de 40px et s'étend à 250px quand l'utilisateur survole ou clique dedans, grâce à la pseudo-classe :focus-within (qui s'active si un enfant a le focus).

3.10 — Gestion des erreurs d'image (onerror)

Pour les images de personnages qui n'existent pas encore sur le disque, un gestionnaire onerror remplace automatiquement l'image manquante par un placeholder généré en ligne :

```
card.innerHTML = `
<img src='${char.img}'
alt='${char.name}'
onerror="this.src='https://placeholder.co/120x150/1a1a1a/888?text=
${encodeURIComponent(char.name)}'">
```

```
<p>${char.name}</p>
```

```
`;
```

Si l'image du personnage est introuvable, le navigateur déclenche l'événement **onerror** et remplace le `src` par une URL de placeholder. **encodeURIComponent()** transforme les espaces et caractères spéciaux du nom (ex: 'Sung Jin-Woo' → 'Sung%20Jin-Woo') pour former une URL valide. Le service placeholder.co génère une image avec le nom en texte.

PARTIE 4 — Bonnes pratiques & pistes d'amélioration

4.1 — Ce qui fonctionne bien

- Script unique partagé → zéro duplication entre les pages
- Event delegation → un seul listener pour toutes les cartes, performant
- Fermeture modale triple (×, fond, Echap) → UX soignée
- onerror sur les images → pas de cartes cassées avec une image manquante
- Correspondance la plus longue → détection précise même avec des titres similaires
- titleToKey séparé des données → gestion des fautes de frappe sans toucher aux données
- Injection CSS/HTML dynamique → applicable à toutes les pages futures automatiquement

4.2 — Points d'amélioration possibles

Typos HTML	Corriger 'Swwordmaster's', 'Drangon Ball', 'Standar Of Reincarnation' dans les fichiers HTML
Sémantique nav	Remplacer les <a> directs dans par des <a>...
Données externes	Déplacer dataManghwa dans un fichier JSON chargé avec fetch() pour séparer données/logique
Recherche globale	Actuellement la recherche filtre les cartes visibles ; elle pourrait chercher aussi dans les synopsis
URL partageables	Permettre de partager un lien vers une oeuvre spécifique (ex: index.html#naruto)
Lazy loading	Ajouter loading='lazy' sur les des cartes pour accélérer le chargement initial
Accessibilité	Ajouter role='dialog', aria-modal='true', aria-labelledby sur le modal pour les lecteurs d'écran
footer manquant	Ajouter le footer sur mangas.html et manhwas.html